

Implementing Linearizability at Large Scale and Low Latency

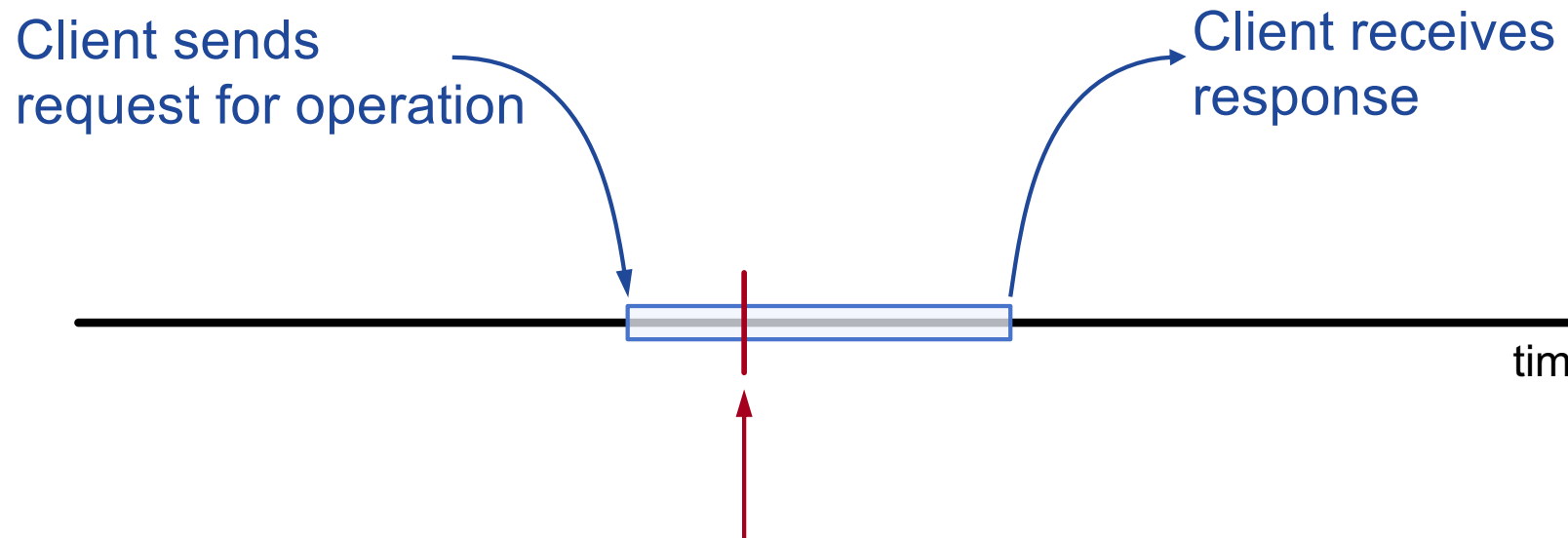
*Collin Lee, **Seo Jin Park**,
Ankita Kejriwal, [†]Satoshi Matsushita,
John Ousterhout*

Platform Lab
Stanford University [†]NEC

Overview

- **Goal:** take back consistency in large-scale systems
- **Approach:** distinct layer for linearizability
- **Reusable Infrastructure for Linearizability (RIFL)**
 - At-least-once RPC → exactly-once RPC
 - Records RPC results durably
 - To handle reconfiguration, associates metadata with object
- **Implemented on distributed KV store, RAMCloud**
 - Low latency: < 5% (500ns) latency overhead
 - Scalable: supports states for 1M clients
- **RIFL simplified implementing transactions**
 - Simple distributed transaction commits in ~22 μ s
 - Outperforms H-Store

What is Linearizability?

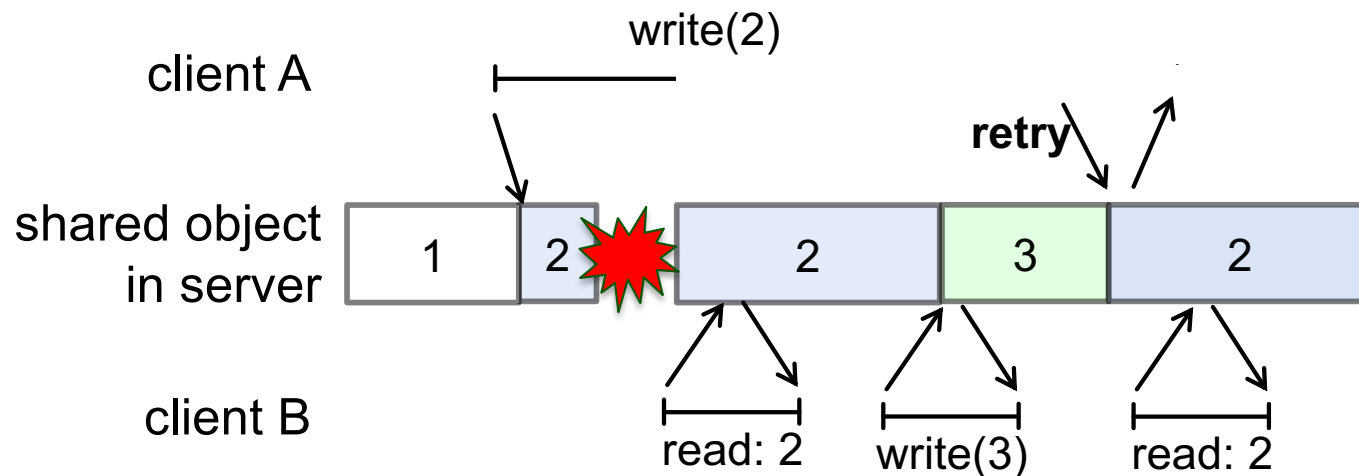


Behaves as if executing exactly once and instantaneously

Strongest form of consistency for concurrent systems

What is Missing from Existing Systems?

- **Most systems: at-least-once semantics, not exactly-once**
 - Retry (possibly) failed operations after crashes
 - Idempotent semantics: repeated executions O.K.?



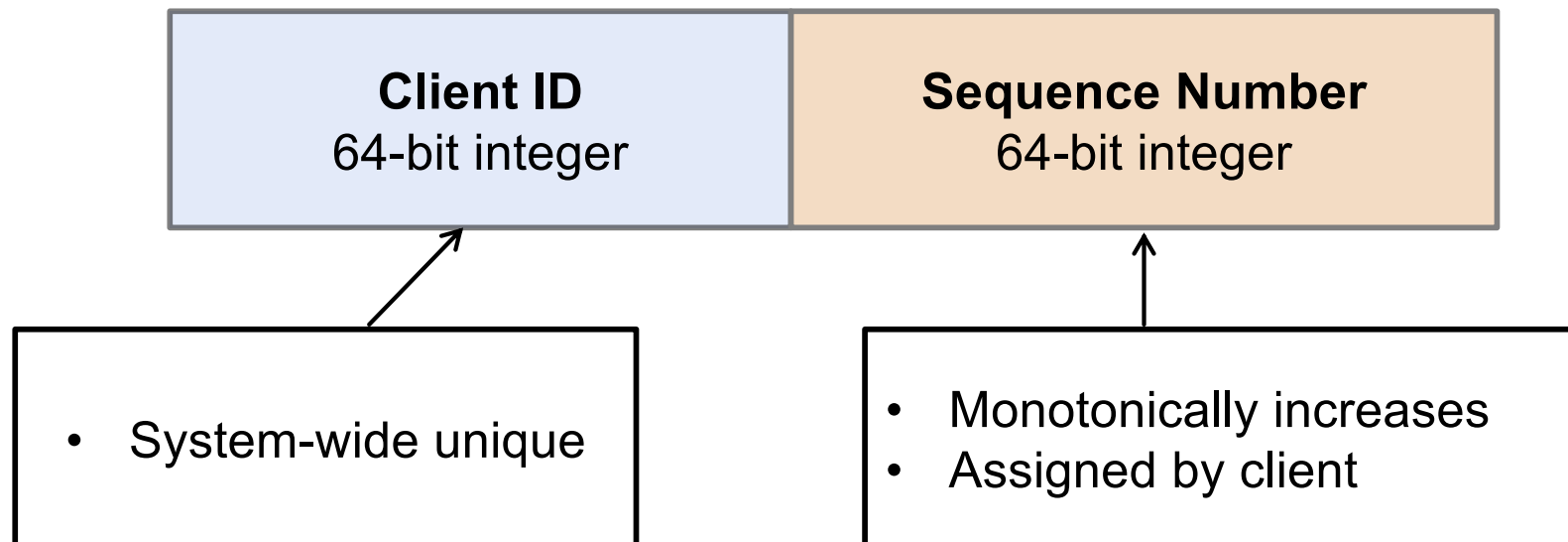
- **At-least-once + idempotency $\neq$ linearizability**
- **Need exactly-once semantics!**

Architecture of RIFL

- **RIFL saves results of RPCs**
- **If client retries,**
 - Don't re-execute
 - Return saved result
- **Key problems**
 - Unique identification for each RPC
 - Durable completion record
 - Retry rendezvous
 - Garbage collection

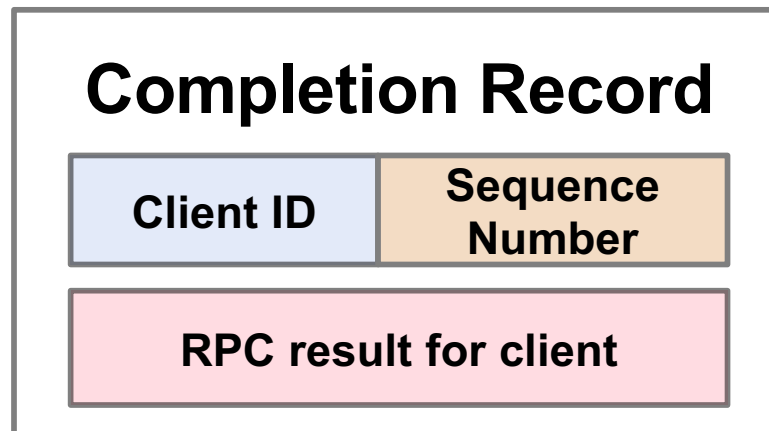
1) RPC Identification

- Each RPC must have a unique ID
- Retries use the same ID
- Client assigns RPC IDs



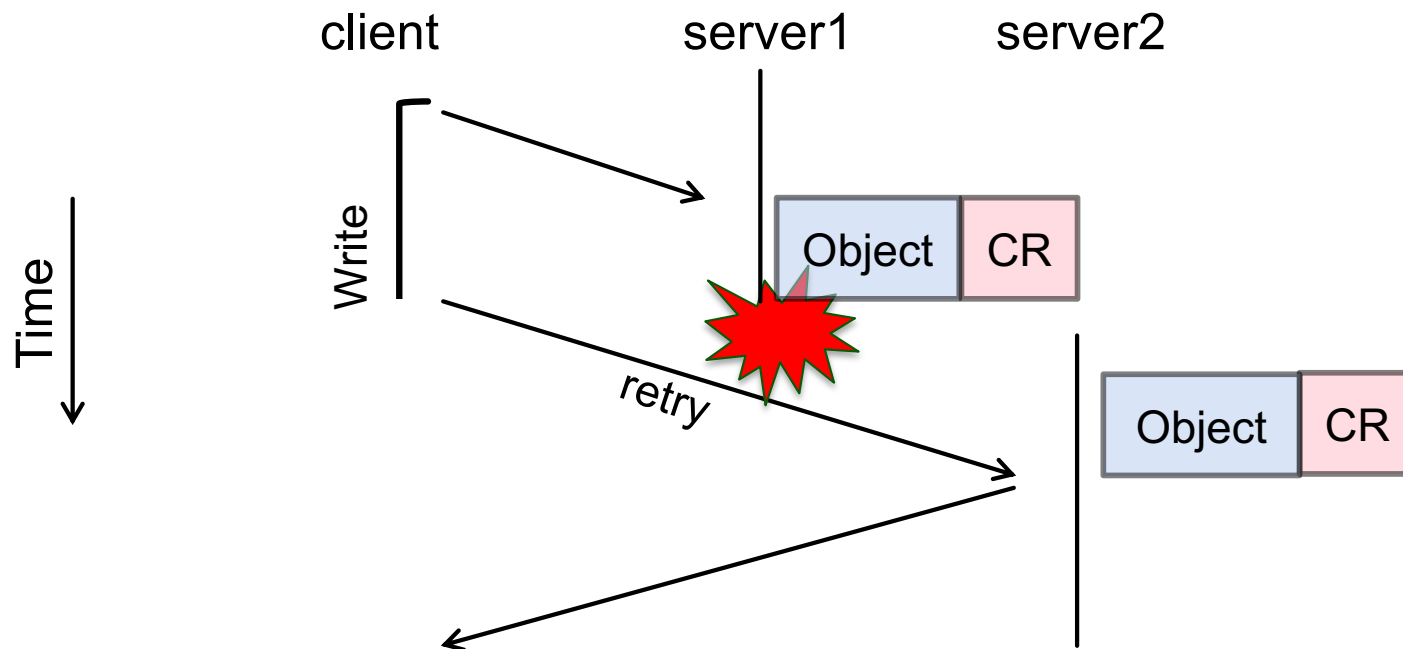
2) Durable Completion Record

- Written when an operation completes
- Same durability as object(s) being mutated
- Atomically created with object mutation



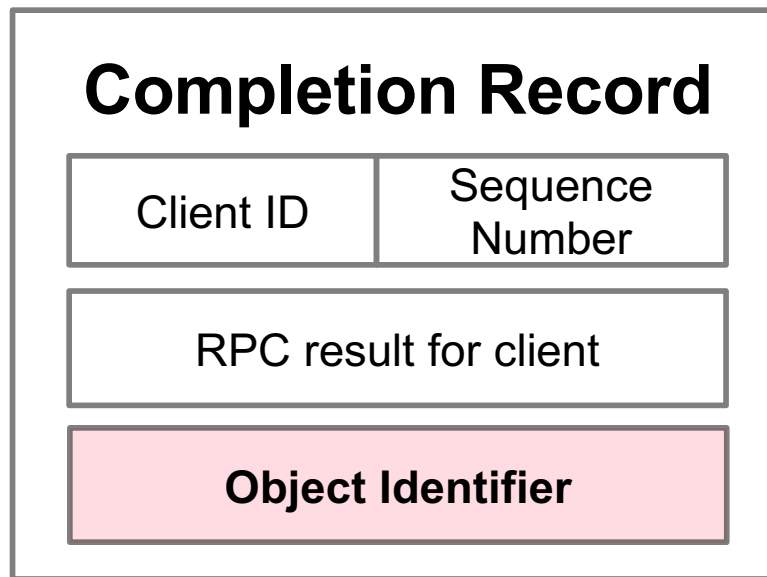
3) Retry Rendezvous

- Data migration is popular in large systems (eg. crash recovery)
- Retries must find completion record



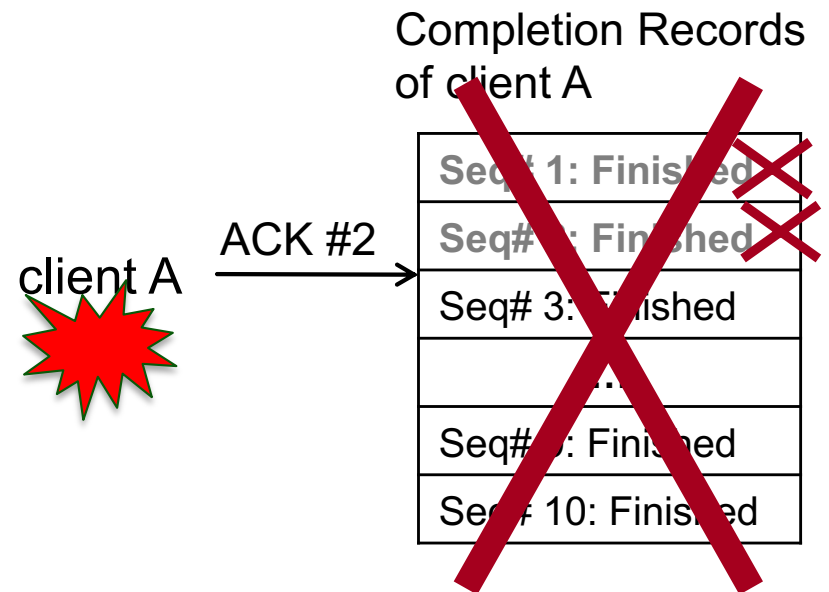
3) Retry Rendezvous (cont.)

- Associate each RPC with a specific object
- Completion record follows the object during migration

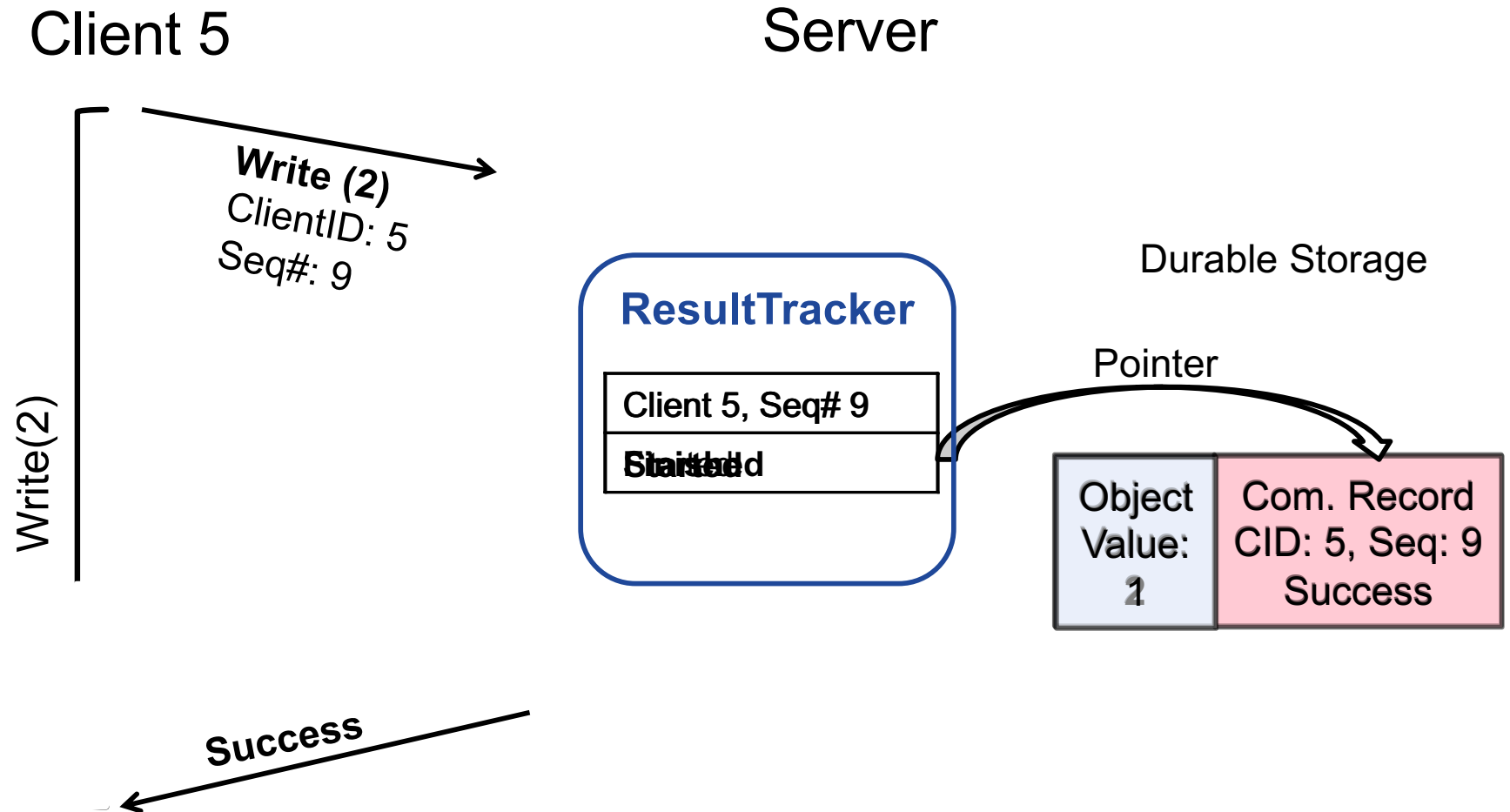


4) Garbage Collection

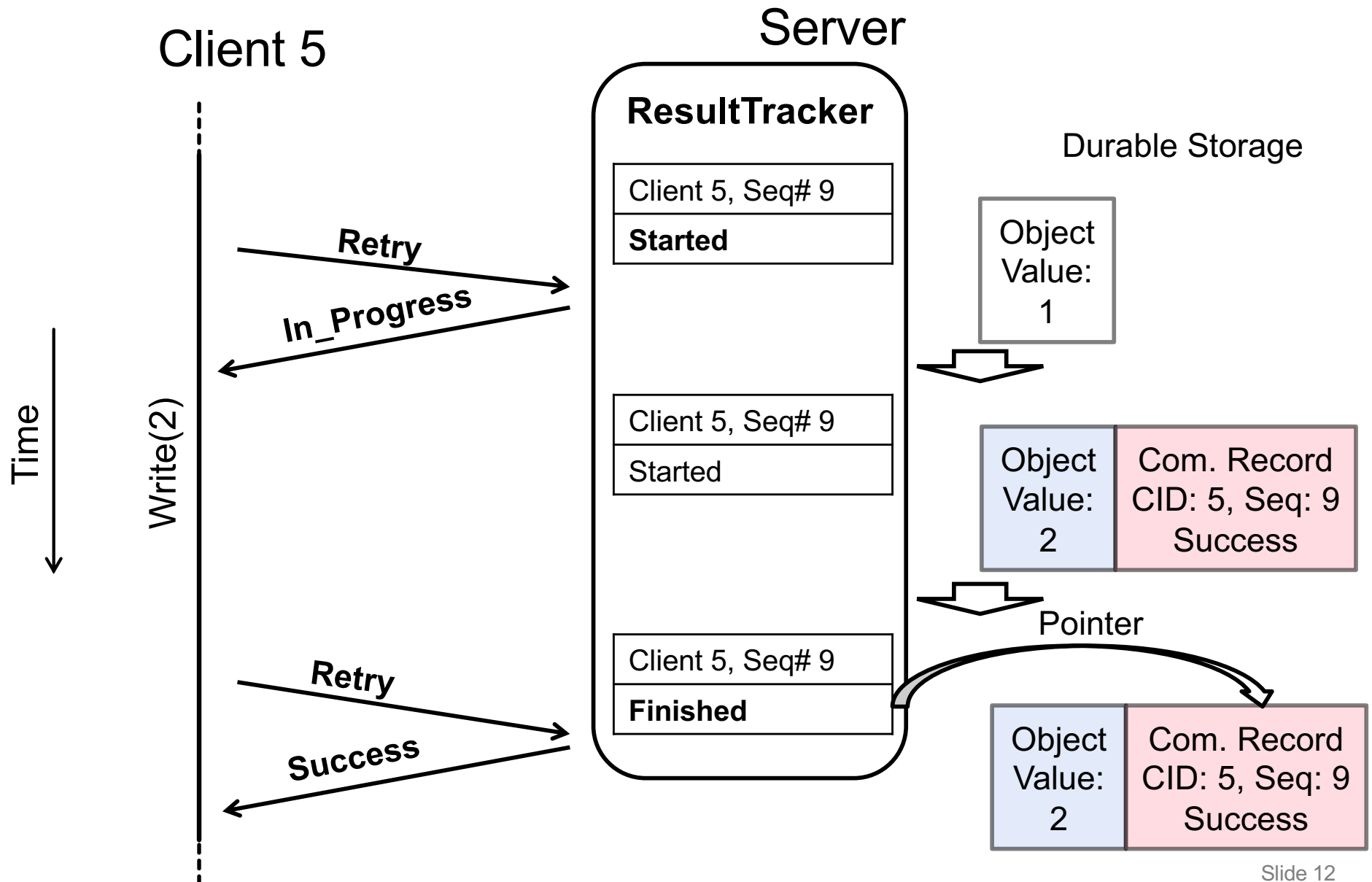
- Lifetime of completion record $\neq$ lifetime of object value
- Can't GC if client may retry later
- Server knows a client will never retry if
 1. Client acknowledges receipt of RPC result
 2. Detect client crashes with lease.



Linearizable RPC in Action



Handling Retries



Performance of RIFL in RAMCloud

- **Achieved linearizability without hurting performance of RAMCloud**
 - Minimal overhead on latency(< 5%) and throughput (~0%)
 - Supports state for 1M clients

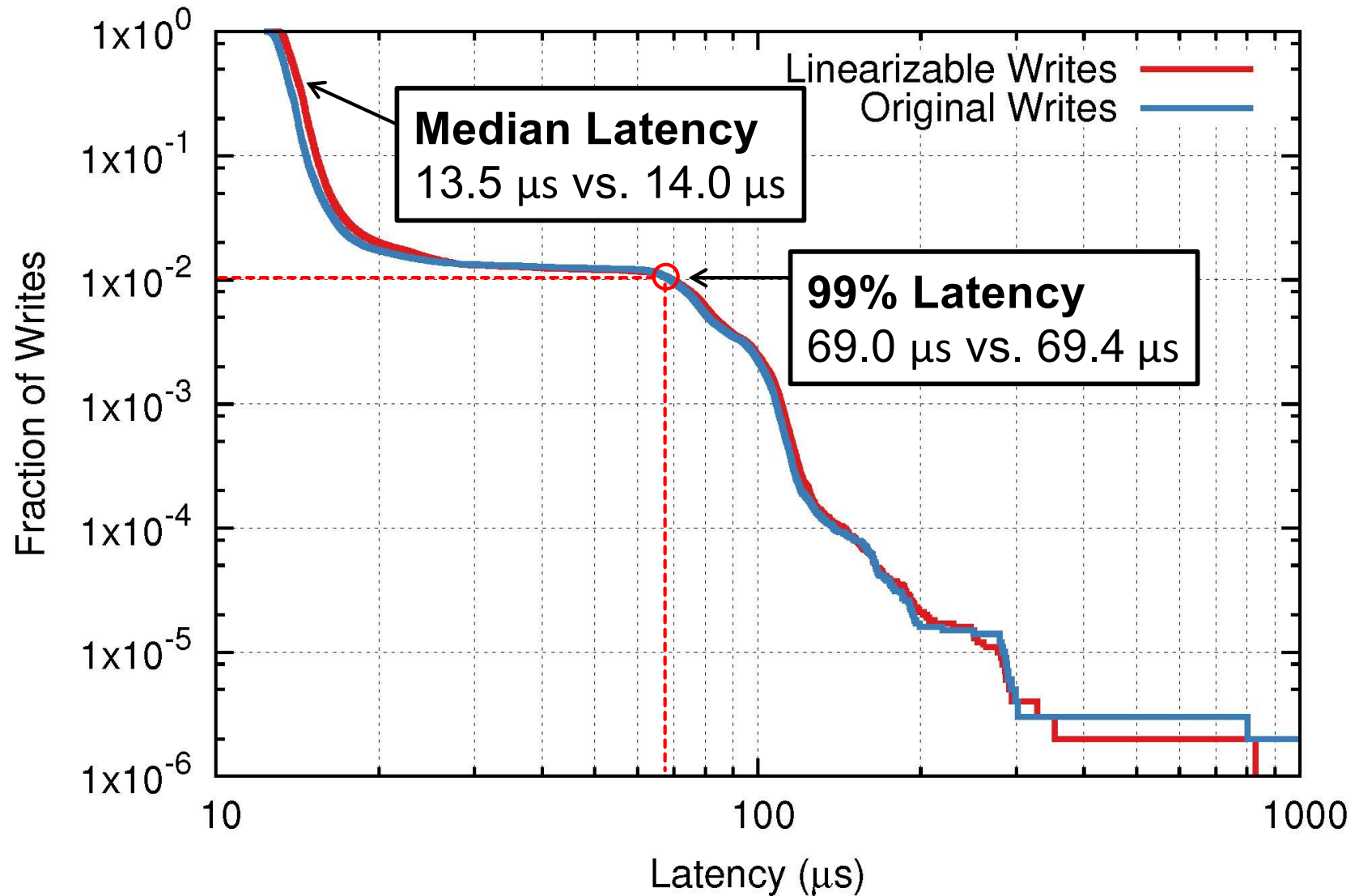
Why RAMCloud?

- **General-purpose distributed in-memory key-value storage system**
- **Durability:** 3-way replication
- **Fast recovery:** 1~2 sec for server crash
- **Large scale:** 1000+ servers, 100+ TB
- **Low latency:** 4.7 μ s read, 13.5 μ s write (100B object)
- **RIFL is implemented on top of RAMCloud**
 - Core: 1200 lines of C++ for infrastructure
 - Per operation: 17 lines of C++ to make an operation linearizable

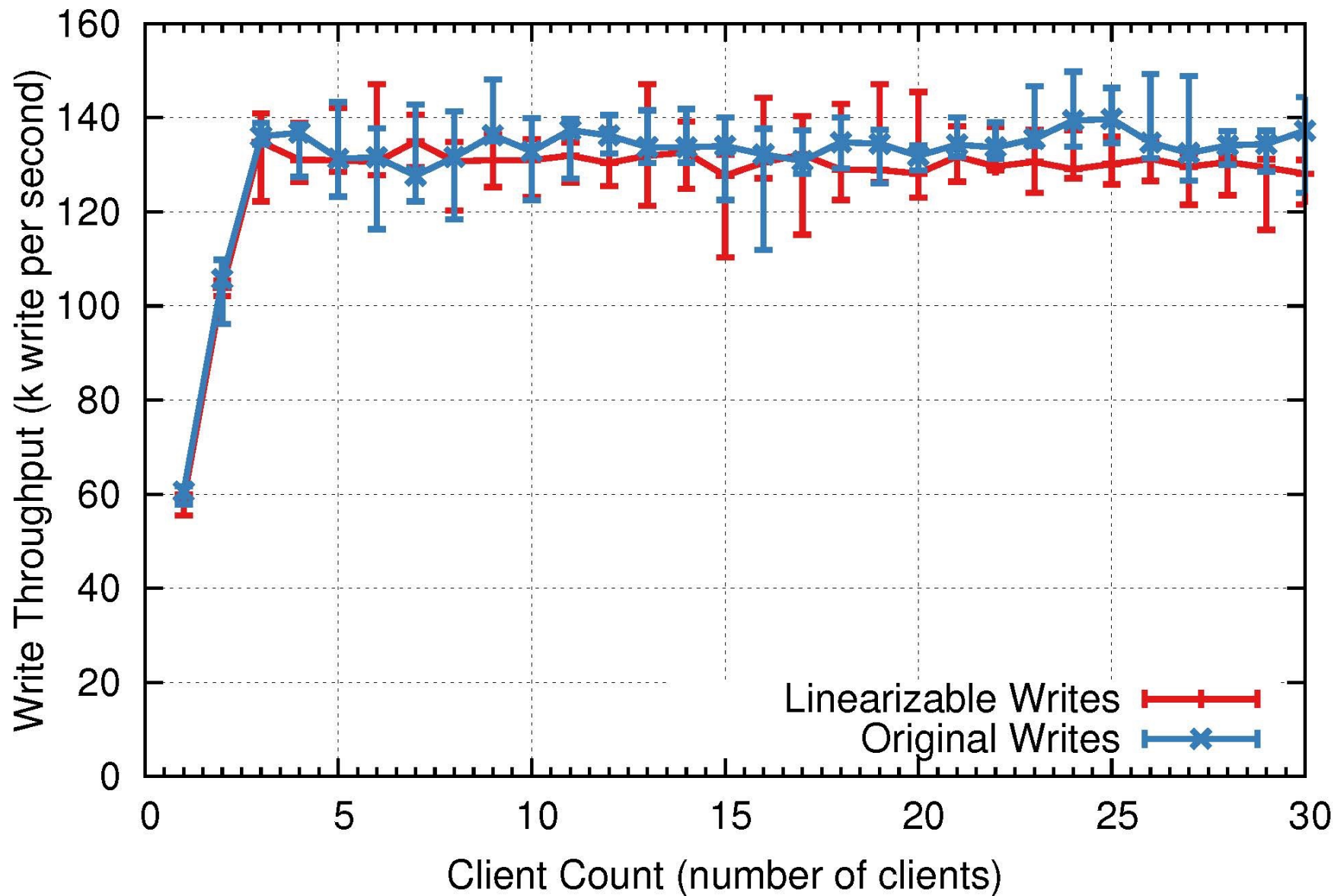
Experimental Setup

- **Server: Xeon 4 cores at 3 GHz**
- **Fast Network**
 - Infiniband (24 Gbps)
 - Kernel-bypassing transport (RAMCloud default)

Impact on Latency



Impact on Throughput



Conclusion

- **Distinct layer for linearizability** → take back consistency in large-scale systems
- **RIFL saves results** of RPCs; If client **retries**, returns saved result **without re-executing**
 - 0.5us (< 5%) latency overhead, almost no throughput overhead.
- **RIFL makes transactions easier**
 - RIFL-based RAMCloud transaction: ~**20** μ s for commit
 - Outperform H-Store for TPC-C benchmark